

Twitch: Learning Abstractions for Equational Theorem Proving

Guy Axelrod^{1,2}[0000–0002–1752–8069], Moa Johansson^{1,2}[0000–0002–1097–8278],
and Nicholas Smallbone^{1,2}[0000–0003–2880–6121]

¹ Chalmers University, of Technology, Gothenburg, Sweden

² University of Gothenburg, Gothenburg, Sweden

{guya,jomoa,nicsma}@chalmers.se

Abstract. Several successful strategies in automated reasoning rely on human-supplied guidance about which term or clause shapes are interesting. In this paper we aim to discover interesting term shapes *automatically*. Specifically, we discover *abstractions*: term patterns that occur over and over again in relevant proofs. We present our tool *Twitch* which discovers abstractions with the help of STITCH, a tool originally developed for discovering reusable library functions in program synthesis tasks. Twitch can produce abstractions in two ways: (1) from a partial, failed proof of a conjecture; (2) from successful proofs of other theorems in the same domain. We have also extended Twee, an equational theorem prover, to use these abstractions. We evaluate Twitch on a set of unit equality (UEQ) problems from TPTP, and show that it can prove 12 rating-1 problems as well as yielding significant speed-ups on many other problems.

Keywords: automated theorem proving · equational theorem proving · completion · abstraction learning · proof-guided search

1 Introduction

Automated theorem provers must navigate a massive search space, choosing the useful inferences and discarding the useless ones. For example, a proof in Twee [17] may consider hundreds of millions of critical pairs, selecting some tens of thousands as active rewrite rules, of which perhaps a few hundred are used in the final proof. Thus, it is essential to identify which critical pairs are interesting – but Twee’s heuristics for this are simplistic, relying mostly on the size of the critical pair. It is hard to do anything smarter because, with so many possible critical pairs, each one must be judged quickly.

This story is typical for saturation-based theorem provers: very many possible inferences, ranked using a simple metric. This is quite effective, but sometimes more intelligence is needed. One approach popularised by Otter [12] is to inject human guidance about which *shapes* of terms or clauses are interesting. The user specifies a set of interesting shapes, and generated clauses that match these shapes are preferred over ones that do not. This idea originated in the *weighting*

strategy [14] and was later refined by the *resonance strategy* [23,3,8] and *hint strategy* [19,20]. All three strategies have been used to find difficult proofs.

In this paper we find interesting term shapes *automatically*. We analyse failed proof attempts and proofs of related conjectures to find term shapes which occur often, which we call *abstractions*. We have added support for abstractions to Twee, and with the help of automatically generated abstractions we are able to prove 12 rating-1 problems from the TPTP [18], as well as several other problems that Twee previously could not solve (see Section 5). Our contributions are:

- A method for automatically generating abstractions from an *unsuccessful attempt* to prove a conjecture (see Section 3.1). The idea is to pick out interesting lemmas from the proof attempt, and find patterns of terms that occur repeatedly in the proofs of those lemmas. To find the patterns we use the tool STITCH [5], which attempts to *compress* a set of terms (here from proofs) by introducing auxiliary function definitions as abbreviations.
- A method for generating abstractions from a corpus of successful proofs in the same domain, again using STITCH (see Section 3.2). The hypothesis is that patterns of terms that occur repeatedly in the proofs are often good abstractions for other conjectures in the same domain. The aim here is to create a methodology inspired by curriculum learning [4] for proving difficult equational problems: first prove some simpler problems in the same domain, abstract out common patterns appearing in those proofs, and use those abstractions to guide the proof of the difficult problem.
- An implementation of abstractions in the equational theorem prover Twee [17], inspired by the weighting strategy [14] (see Section 4).

The code for Twitch as well as all data and experimental results are supplied in the associated repository [1].

1.1 Motivating Example

Let us consider the problem LAT075-1 from the TPTP, which concerns the equational axiomatization of modular ortholattices in a language with a single function symbol f representing the Sheffer stroke (or NAND operator). The details of this problem are not important here, but what is relevant is that the same kind of term appears repeatedly in the proof that Twee finds. Here is a short fragment of the proof:

$$\begin{aligned}
& \vdots \\
& = f(x, f(y, \boxed{f(f(z, \boxed{f(x, x)}), f(z, \boxed{f(x, x)}))})) \\
& = f(x, f(y, f(f(z, \boxed{f(x, x)}), f(\boxed{f(x, x)}, z)))) \\
& = f(x, f(y, \boxed{f(f(\boxed{f(x, x)}, z), f(\boxed{f(x, x)}, z))}))
\end{aligned}$$

$$\begin{aligned}
&= f(\boxed{f(\boxed{f(x, x)}, \boxed{f(x, x)})}, f(y, \boxed{f(f(\boxed{f(x, x)}, z), f(\boxed{f(x, x)}, z))})) \\
&\vdots
\end{aligned}$$

As we can see, terms of the form $f(t, t)$, which we have marked with a box above, appear over and over in the proof.³ Our hypothesis is that *the same term shape* is likely to be common in proofs of other theorems about the Sheffer stroke.

This gives us the following basic approach to proving a difficult theorem. First we analyse a proof of a related theorem to find commonly occurring term patterns. Then, on the assumption that these term patterns should also occur in the proof of the difficult theorem, we ask the theorem prover to prefer inferences involving those term patterns. The remaining questions are, how can we find these term patterns, and how can the prover exploit them?

To formalize the notion of *common patterns* in a proof, we imagine *compressing* the proof by introducing new definitions to abbreviate certain terms. For example, if we were to define $g(x) := f(x, x)$, then the proof above could be written more compactly; e.g. the first term could be written as $f(x, f(y, g(f(z, g(x)))))$. Then we ask: what is a *maximally compressive* set of abbreviations, that is, a set of definitions that allows us to express the proof terms as compactly as possible?

The tool STITCH [5] (see Section 2) is designed to answer exactly this question. Given a set of terms, it searches for the maximally compressive function definitions. Henceforth, we will call such function definitions *abstractions*. In this case, STITCH would find that $g(x) := f(x, x)$ is a maximally compressive abstraction (and when given the full proof, it also finds other useful abstractions, such as the AND operator $h(x, y) := f(f(x, y), f(x, y))$).

Once we have these abstractions, how can we use them to influence proof search? One possibility is to simply add the function definition as an axiom, in this case $\forall x. g(x) = f(x, x)$. This would allow the prover to rewrite terms of the form $f(t, t)$ to $g(t)$, which has a reduced cost as it is smaller, so that if such a term occurs in a critical pair then the critical pair is more likely to be selected. For LAT075-1 this approach works well, and reduces the runtime of Twee from $\sim 250s$ to $\sim 10s$.

Adding definitional axioms that abbreviate common term patterns is a known technique that often works well.⁴ However, adding new axioms also allows new inferences between them, thus expanding the search space. As we show in Section 5.3, adding more than a few such axioms harms the prover’s performance.

Instead, we have extended Twee to natively support abstractions (see Section 4). The abstractions influence the heuristic function of the prover, so that inferences matching them are more likely to be selected, but they do not alter the search space itself. Because of this, we can provide many more abstractions, but the individual abstractions may have a smaller effect. In this case, passing $f(x, x)$ as an abstraction reduces the runtime of Twee from $\sim 250s$ to $\sim 130s$.

³ This is perhaps not so surprising as $f(t, t)$ represents the negation of t .

⁴ For example, see the Robbins problem variant ROB033-1 in the TPTP. It is also the basis of Twee’s goal direction heuristic [17].

2 Background: STITCH

STITCH [5] is a system originally designed for library learning in program synthesis – automatically discovering reusable function abstractions from an existing collection of programs. Given a set of terms as input, it constructs functions which generalize parts of the terms, and uses those functions to abbreviate the terms. STITCH attempts to construct *maximally compressive* abstractions; specifically, it maximizes the product of the size of the function and the number of places where the function can be used.

Although STITCH was designed to work on program code, it accepts as input any set of terms. Hence, given a Tweak proof, we can give STITCH the set of all terms occurring in the proof and it will find abstractions for it.

STITCH’s input language is a higher-order λ -calculus. Thus, we need to translate proof terms into λ -terms, which we do by treating function symbols as constants, and λ -abstracting all free variables. For example, the term $f(x, f(y, f(f(z, f(x, x)), f(z, f(x, x))))$ from our example is translated into $\lambda x.\lambda y.\lambda z. (f\ x\ (f\ y\ (f\ (f\ z\ (f\ x\ x))\ (f\ z\ (f\ x\ x))))$. In this case, STITCH will return the abstraction $g(\alpha) := (f\ \alpha\ \alpha)$, where α is a *schematic parameter* standing for an arbitrary subterm, which can be used to compress the input term to $\lambda x.\lambda y.\lambda z. (f\ x\ (f\ y\ g(f\ z\ g(x))))$. Because the body of g has a first-order shape, it can be translated back directly, yielding the abstraction $f(x, x)$.

In general, STITCH abstractions are arbitrary λ -terms and can be higher-order. In practice, STITCH rarely generates higher-order abstractions in our setting since the input terms are first-order. If it does, we discard the abstraction.

3 Tweak + STITCH = Twitch

In this section we present the implementation of Twitch: a system for automating the discovery of reusable abstractions that can be used to help solve hard problems in a given domain. Twitch has two modes of usage: abstractions can be discovered from partial proof attempts as described in section 3.1, or from easier proofs in the same domain, as described in section 3.2. We say that the first mode produces *partial proof abstractions*, while the second produces *domain abstractions*. An overview of how domain abstractions are derived is shown in Figure 1. The abstractions found by Twitch are used to adjust the search heuristics in the Tweak prover, as described in Section 4.

Henceforth, we denote a given input problem – which consists of a set of equations (axioms) and a goal/conjecture equation to prove – by P .

3.1 Abstractions from Partial Proofs

The first approach we explore discovers abstractions given as input a single hard problem P_h , and can be used when we do not have examples of related problems to learn from. We start by running Tweak (without abstractions) on P_h for a fixed amount of time. When it times out, even though it has not proved the goal, it will

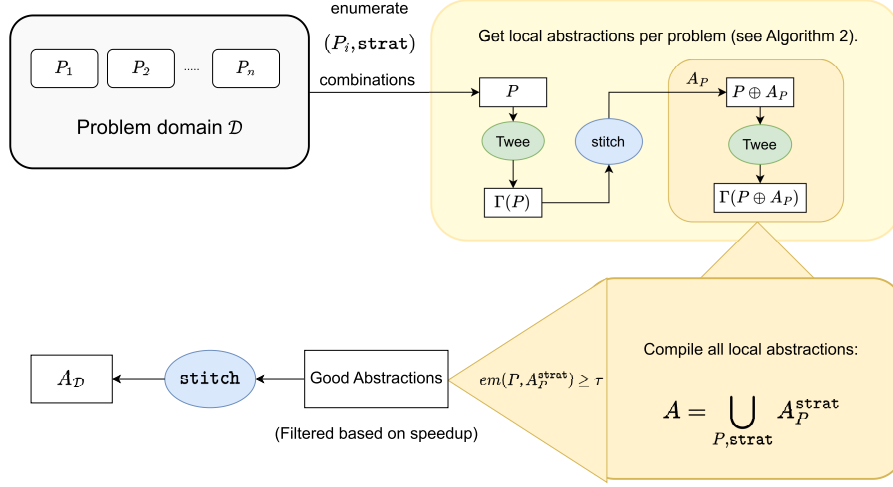


Fig. 1: Overview of domain abstraction generation via Twitch (Algorithm 1 and its subroutine Algorithm 2).

have derived a large collection of rewrite rules. Each of these derived rules $l \rightarrow r$ corresponds to an equation $l = r$ about the theory we are exploring. Many of these equations will be quite boring, but some will represent interesting lemmas. Our idea is to learn abstractions from only the most interesting lemmas.

In order to define interestingness, we first need some auxiliary definitions. For any lemma $l = r$, we shall denote by $T(l = r)$ the set of terms occurring in its proof, including the proofs of any sub-lemmas used in the proof. More precisely, the proof of $l = r$ is a chain $l = t_1 = \dots = t_n = r$, where each step may be justified by a lemma $l_i = r_i$ having its own proof. We then define $T(l = r) = \{l, t_1, \dots, t_n, r\} \cup \bigcup_i T(l_i = r_i)$.

For a term t , we define its weight $|t|$ as the number of function applications plus the number of unique variables occurring in t . We extend this to equations by defining $|l = r| = |l| + |r|$, and to sets of terms T by defining $|T| = \sum_{t \in T} |t|$. We then define the “interestingness” $s(l = r)$ of a lemma as follows:

$$s(l = r) := \frac{|T(l = r)|}{|l = r|^2}.$$

The intuition is that lemmas with higher scores are those that are simple statements requiring a long proof. Of course this does not necessarily coincide with what a human would consider interesting or useful, but we take it as a rough heuristic towards measuring these notions.

Given some partial proof of P_h , we sort the lemmas discovered in decreasing order of s . We then extract the first k lemmas $l_1 = r_1, \dots, l_k = r_k$. What we call the *partial proof abstractions* are simply those we get by running STITCH on $\bigcup_{1 \leq i \leq k} T(l_i = r_i)$. We can then rerun Twee on P_h augmented with these abstractions.

3.2 Domain Abstractions

The goal of this approach is to learn abstractions that are useful for proving problems in a particular domain $\mathcal{D}$. We require a set of problems, some of which are already proved (the “easy” problems) and some of which are not (the “hard” problems). The algorithm works as follows:

1. First, we use STITCH on the proofs of the easy problems to find abstractions for each problem. These abstractions are *local*, i.e. specific to one problem.
2. Then, we augment each problem with its local abstractions and re-run Twee to empirically establish if the abstractions are *good*, i.e. if they allow us to reprove the problem faster.
3. Finally, we build a set of all the good local abstractions, and compress it with STITCH to find commonalities and derive higher-level *domain abstractions*.

This process is illustrated in Figure 1, with pseudocode in Algorithm 1. There, we use $T(\Gamma(P))$ to denote the set of terms occurring in the proof $\Gamma(P)$ of problem P found by Twee. The set of abstractions derived by running STITCH on $T(\Gamma(P))$ (where $\Gamma(P)$ was found using a specific Twee strategy **strat**) is denoted by A_P^{strat} . We use $P \oplus A$ to denote P augmented with abstractions A . Finally, the local abstractions are filtered by an effect metric em which measures the speedup obtained by augmenting the problem with the abstraction. More precisely,

$$em(P, A) = \frac{t(\Gamma(P))}{t(\Gamma(P \oplus A))},$$

where $t(\Gamma(P))$ denotes the time taken by Twee to find proof $\Gamma(P)$. Only those abstractions that lead to a speedup above some threshold τ are added to the set of good abstractions.

We see that growing the good abstraction set relies on running the LOCAL-ABSTRACTIONS subroutine for a collection of different *strategies*. This simply refers to an assignment of specific values to parameters that influence the working of Twee. In particular, the strategy parameters are (1) whether to run Twee

Algorithm 1

```

1: procedure DOMAINABSTRACTIONS( $\mathcal{D}$ )
2:   good_absts  $\leftarrow$  empty set
3:   for all combinations of  $P \in \mathcal{D}$  and strat  $\in$  strategies do
4:      $A_P^{\text{strat}}, em(P, A_P^{\text{strat}}) \leftarrow \text{LOCALABSTRACTIONS}(P, \text{strat})$ 
5:     if  $em(P, A_P^{\text{strat}}) \geq \tau$  then
6:       Add the alpha-normalized  $A_P^{\text{strat}}$  to good_absts
7:     end if
8:   end for
9:    $\mathcal{D} \leftarrow \text{Run STITCH over good\_absts}$ 
10:  Return  $\mathcal{D}$ 
11: end procedure

```

Algorithm 2

```

1: procedure LOCALABSTRACTIONS( $P$ ,  $\mathbf{strat}$ )
2:   Run Twee on  $P$  with  $\mathbf{strat}$ .
3:   If it fails to terminate within the time limit, return failure.
4:   Else we get proof  $\Gamma(P)$ .
5:   Extract terms from  $\Gamma(P)$  as strings to get list  $T(\Gamma(P))$ .
6:   Run STITCH on  $T(\Gamma(P))$  to get abstractions  $A_P^{\mathbf{strat}}$ .
7:   Run Twee on  $P \oplus A_P^{\mathbf{strat}}$  with  $\mathbf{strat}$ .
8:   Compute  $em(P, A_P^{\mathbf{strat}})$ . If Twee timed out on  $P \oplus A_P^{\mathbf{strat}}$ , set  $em(P, A_P^{\mathbf{strat}}) = 0$ .
9:   Return  $A_P^{\mathbf{strat}}$ ,  $em(P, A_P^{\mathbf{strat}})$ .
10: end procedure

```

with or without goal flattening and (2) the abstraction weight factor (see Section 4.1). By iterating over combinations of these parameters, we increase the pool of good abstractions from which to derive domain abstractions later.

We can then use these domain abstractions to augment hard problems in the same domain (see Section 5.2). In practice, given a hard problem $P_h \in \mathcal{D}$, we perform a filtering step on the good abstractions before running STITCH for the second time (i.e. before line 10 in Algorithm 1). Specifically, we remove any abstraction that contain function symbols not present in P_h , or shared function symbols with mismatching arities. This ensures that the resulting domain abstractions are compatible with P_h .

4 Supporting Abstractions in Twee

Twee’s architecture is based on *unfailing completion* [11,2], which works by treating the input equations as rewrite rules and then deriving new rules that cannot be proved by forward rewriting. For example, given some axioms about group theory, Twee might orient them into the following rules:

$$e \cdot x \rightarrow x \quad x^{-1} \cdot x \rightarrow e \quad (x \cdot y) \cdot z \rightarrow x \cdot (y \cdot z)$$

We can observe that the term $(x^{-1} \cdot x) \cdot y$ has two normal forms:

$$(x^{-1} \cdot x) \cdot y \rightarrow e \cdot y \rightarrow y \quad (x^{-1} \cdot x) \cdot y \rightarrow x^{-1} \cdot (x \cdot y),$$

hence the equation $x^{-1} \cdot (x \cdot y) = y$ is true but cannot be proved by forward rewriting, since both sides are normal forms (this is called a *critical pair*). Twee therefore adds the new rewrite rule $x^{-1} \cdot (x \cdot y) \rightarrow y$. This process of deriving new rewrite rules continues until the rule set is saturated or the goal is proved.

One problem is that the number of critical pairs typically grows quadratically in the number of derived rules: a typical proof may produce $\sim 10^5$ rules with $\sim 10^{10}$ possible critical pairs. Hence, we cannot turn *all* critical pairs into rules. Twee instead uses a heuristic approach where, at each step, the “best” critical pair is turned into a rewrite rule. Twee uses several criteria to rank the critical pairs (see [17]) but the most important one is the equation’s *weight*, defined as:

$$\begin{aligned}
w(t = u) &= w(t) + w(u) \\
w(x) &= 1, \text{ if } x \text{ is a variable} \\
w(F(t_1, \dots, t_n)) &= 1 + \sum_{i=1}^n w(t_i), \text{ if } F \text{ is a function symbol}
\end{aligned}$$

Our approach to abstractions is inspired by the *weighting strategy* [14]: whenever a term or subterm in a critical pair matches an abstraction, reduce its computed weight. Specifically, we extend w with the following clause:

$$w(A\sigma) = w_A + \sum_{x \in \text{dom}(\sigma)} w(x\sigma), \text{ if } A \text{ is an abstraction and } \sigma \text{ is a substitution}$$

Here, w_A is a constant representing the weight of the abstraction. We come back to this in Section 4.1, but for now, let us assume that $w_A = 1$.

For example, suppose we are studying two functions *times* and *plus*. To encourage the prover to reason about distributivity, we might give the following abstraction A , representing a term in which distributivity can be applied.

$$\text{plus}(\text{times}(x, y), \text{times}(x, z))$$

Now consider the following equation, perhaps occurring as a critical pair:

$$\text{times}(c, \underline{\text{plus}(\text{times}(g(f(a), b), f(x)), \text{times}(g(f(a), b), g(y, z)))}) = g(a, b)$$

Without the abstraction A , this equation would have a weight of 21. With A , the underlined term can be matched as $A\sigma$, where $\sigma = \{x \mapsto g(f(a), b), y \mapsto f(x), z \mapsto g(y, z)\}$. Hence the total weight is $1 + w(c) + w(A\sigma) + w(g(a, b)) = 2 + w_A + w(g(f(a), b)) + w(f(x)) + w(g(y, z)) + 3 = 15$. This means that, if this equation is produced as a critical pair, it is more likely to be selected.

Note that using A reduces the cost of the equation above in two ways: (1) the “skeleton” of the abstraction, $\text{plus}(\text{times}(_, _), \text{times}(_, _))$, only counts for weight 1 instead of 3; (2) although x occurs twice in the abstraction, $x\sigma$ is only counted once, hence the repeated subterm $g(f(a), b)$ is only counted once. Property (2) is vital in order for abstractions to act like *abbreviations*. For example, in Section 1.1 we define the abstraction $f(x, x)$, where f is the Sheffer stroke, which represents the negation of x . Hence we should like $f(t, t)$ to have the same weight as a hypothetical term $\text{not}(t)$, which requires property (2).

As an aside, we have also implemented term-level *resonators* [23] in Twee. These work just the same as abstractions, except that they only match if every variable in σ is mapped to a variable (not a compound term).

4.1 The Cost of an Abstraction

In the example above, we assumed that the weight w_A of an abstraction was 1 symbol. In reality, we allow the user to choose between two methods of computing the weight. Both methods are parametrized on a constant k chosen by the user:

Constant weight Each abstraction A gets a fixed weight $w_A = k$.

Weight factor This method gives larger abstractions greater weight. Given the abstraction A , we first compute the weight of A 's *skeleton*, $w(\text{skel}(A))$. This is the weight of A if all variables are counted as weight 0 (i.e. ignored). In the example above, $w(\text{skel}(A)) = 3$. Then we set $w_A = w(\text{skel}(A)) \times k$.

5 TPTP Experiments

We have evaluated Twitch on a set of unsatisfiable unit equality (UEQ) problems from TPTP v9.2.1 [18]. The problems are drawn from nine TPTP domains, summarized in Table 1. The main purpose is to measure whether the abstractions discovered by Twitch help in solving problems from the TPTP.

Section 5.1 measures whether partial proof abstractions and domain abstractions improve Twee's runtime. For domain abstractions, the user must define which problems make up the domain; in our experiments (with one exception mentioned in Section 5.2), we simply considered the domain to be the set of problems in a given TPTP domain (e.g. Group Theory (GRP), Lattice Theory (LAT), etc.)

Section 5.2 reports on the hard problems that abstractions from Twitch allow us to solve. Here, we define a problem as hard if it has TPTP rating at least 0.9 (i.e. 90% of ATPs cannot solve it within 300 seconds) and Twee (without abstractions) cannot solve it within 1000 seconds. There are 70 such problems from Table 1.

Section 5.3 considers various ablations testing some aspects of Twitch's architecture and strategies.

For details regarding relevant parameter values used, see Appendix A.

5.1 Overall Runtime Improvements

Figures 2a and 2b include cactus plots of runtimes of Twee on the TPTP problems listed in Table 1. We benchmark Twee both with and without its *goal*

Domain	Count
ALG (General Algebra)	20
BOO (Boolean Algebra)	56
COL (Combinatory Logic)	120
GRP (Group Theory)	481
LAT (Lattice Theory)	126
LCL (Logic Calculi)	84
REL (Relation Algebra)	81
RNG (Ring Theory)	48
ROB (Robbins Algebra)	25
Total	1041

Table 1: Unsatisfiable UEQ problems per TPTP domain, TPTP v9.2.1

flattening strategy [17], in order to see how this strategy interacts with abstractions. In both figures, the ‘Twee (flat)’ and ‘Twee (no flat)’ plots show Twee running without abstractions (with and without goal flattening respectively) for the full time limit of 1000s. We have plotted a line at 300s, a typical timeout for ATP benchmarking, but continue the evaluation until 1000s.

Figure 2a shows the results for partial proof abstractions. Here, we first run Twee (without goal flattening) for 150 seconds, then if it times out, extract partial proof abstractions from the failed proof, and rerun Twee (with goal flattening) augmented with these abstractions for the remaining time.

Figure 2b shows the results when using domain abstractions. As before, the ‘Twee (flat)’ and ‘Twee (no flat)’ show the performance of Twee without abstractions. ‘Absts. (flat)’ and ‘Absts. (no flat)’ show the performance of Twee once the abstractions are added. Unlike with partial proof abstractions, there is no time-slicing. The plot does not include the time to learn the abstractions for a given domain, as this is shared between all problems in the domain.

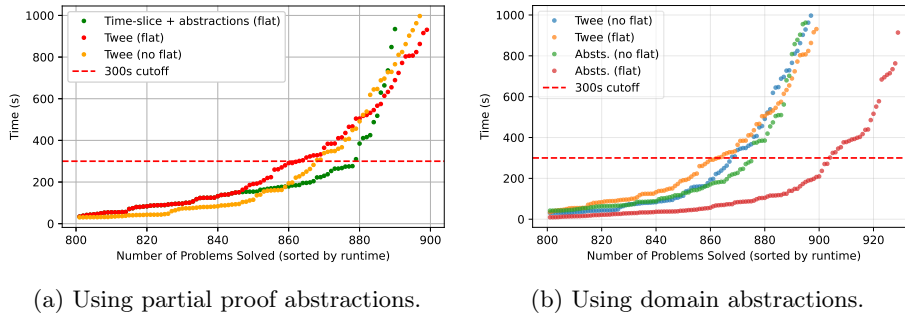


Fig. 2: Cactus plots of Twee runtimes for TPTP problems. A given point in these plots corresponds to a particular problem P from Table 1. The y-value of a point indicates the time taken by Twee to find a proof of P (possibly augmented with Twitch domain abstractions), and the corresponding x-value is the index of P when the problems are sorted in increasing order of runtime.

We see that applying just domain abstractions to a problem results in a modest speedup. However, applying both domain abstractions and goal flattening leads to a large speedup, with roughly 25 more problems solved inside of 300 seconds. When comparing only the problems that baseline Twee could solve within 300 seconds, adding domain abstractions with goal flattening roughly halves the runtime. Hence domain abstractions seem to interact particularly well with goal flattening.

We see that partial proof abstractions do provide some improvement in runtimes, and solve more problems after 300s, but are not as effective as domain abstractions. However, they are still useful in many cases since they do not require any pre-processing to find the abstractions. Furthermore, they have the

advantage of being available for any problem, even those for which we have no other proofs to learn from.

5.2 Solving Hard Problems

Along with general runtime improvements, we find that adding Twitch abstractions allows us to prove 18 hard problems. The full list of hard problems solved can be found in Appendix C. We give a summary of the results here, and highlight some interesting cases.

Partial Proof Abstractions Given a hard problem, we run Twee on it for at most 500 seconds. We then generate abstractions from the top lemmas of the partial proof, and supply these to Twee. This allowed us to solve 11 hard problems – each with a total time limit of 1000 seconds, including the partial proof run. The majority of these (6 out of the 11) are LCL problems that contain translations of Horn clauses into equational logic (using the technique introduced in [7]). In these cases, we find that partial proof abstractions of the form $ifeq(x, true, y, true)$ are always discovered – encodings of implication in terms of the $ifeq$ function. And through manual testing, we find that these are the abstractions that make the real difference in helping solve the hard problems.

Domain Abstractions Domain abstractions solved 18 hard problems within 1000 seconds, including every hard problem solved by partial proof abstractions. For instance, the problem LAT075-1 (Rating: 1.00) concerning a Sheffer stroke axiomatization was solved in 32 seconds (without goal flattening) by providing the following domain abstractions:

- $f(A, A)$
- $f(f(B, A), f(B, A))$
- $f(f(A, B), f(A, C))$
- $f(f(D, C), f(B, A))$
- $f(f(f(C, f(C, f(f(A, A), B))), B), A)$

Checking manually, we found that the first and third abstractions were crucial to finding a proof in a reasonable amount of time.

We note that at least the first three abstractions are semantically meaningful. The first one corresponds to the *not* operator, and the second one to *and*. The Sheffer stroke satisfies a kind of self-distributivity law (in which a *nand* distributes over a *nand* to create a *nor*) and the third time represents a term to which this can be applied.

Further inspection of the full results (supplied in the source repository) shows that the abstraction list are quite long in a few cases (three to be exact). These cases highlight the fact that in certain scenarios, it is helpful to skip abstracting over the good abstractions (line 10 of Procedure 1) and simply take $A_{\mathcal{D}}$ to be the set of all good abstractions.

We also note that, in addition to the 18 problems, Problem GRP677-1.p was also solved (in 180 s) using domain abstractions. However, this required constraining the domain $\mathcal{D}$ to only include problems that share very similar axioms to the hard problem being solved. So, we see that a larger domain is not necessarily better. This suggests that, when aiming to solve a particular hard problem, curation of the domain we provide Twitch may be a useful direction for future work.

5.3 Abstractions vs Definitional Axioms

There are two ways one might use STITCH abstractions to augment a problem P – by giving them to Twee as abstractions, or by adding them as definitional axioms (as described in Section 1.1). Twitch uses the abstractions approach. We here present experiments that aim to validate this choice, by comparing the two approaches. The experiments involved running Algorithm 2 on the TPTP problems listed in Table 1.

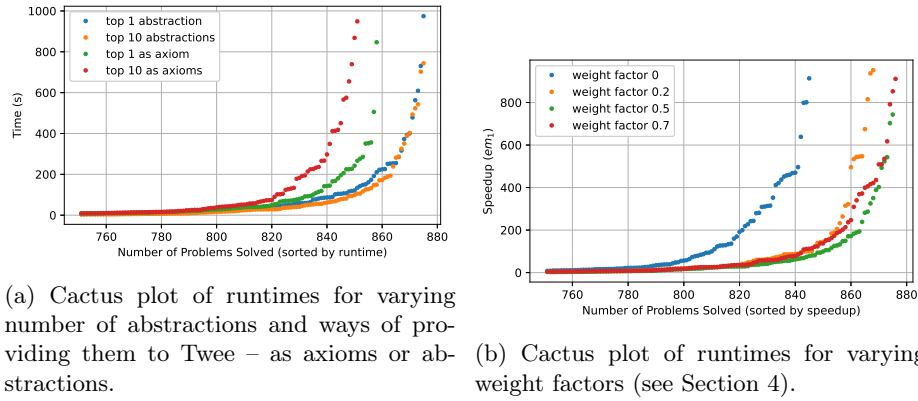
Figure 3a depicts cactus plots of runtimes obtained in line 7 of Algorithm 2 for each problem. The different colours correspond to different approaches in augmenting P with A . The ‘top k ’ in the figure legends refers to taking the top k abstractions found by STITCH in line 6 of Algorithm 2.⁵

We see that overall abstractions lead to lower runtimes compared to definitional axioms, and in particular, fewer cases in which Twee times out as a result of the augmentation. As such, the abstractions approach provides a larger pool of good abstractions for Twitch to derive domain abstractions from. Furthermore, we see that increasing from 1 to 10 abstractions leads to a significant increase in timeouts when using axioms, while the abstractions approach is much more robust to this increase. Hence, using the domain abstractions derived from Twitch as Twee abstractions to augment some hard problem allows us to provide more abstractions with a lesser chance of incurring substantial slowdowns.

However, there are still some problems where providing abstractions as axioms lead to more significant speedups compared to using Twee’s abstraction mechanism. For example, our earlier motivating LAT075-1 problem illustrates such a case.⁶ This suggests that there may be room for combining the two approaches in future work.

⁵ Note that we do not necessarily actually use all k abstractions to augment the problem, as some may not be back-translatable (see translation discussion in Section 2). Nonetheless, the comparison remains informative as the actual number of abstractions used for a given problem during this experiment was the same for both the abstraction and axiom approaches.

⁶ There are also a few problems (15 to be exact) where axioms lead to speedups while abstractions lead to slowdowns. With P taken as LAT141-1, we get a proof $\Gamma(P)$ from default Twee (without goal-flattening) in ~ 160 seconds. The top abstraction found by STITCH on this proof is $f(X, Y, Z) = \text{meet}(X, \text{join}(Y, Z))$. Adding this as an axiom leads to a proof of $P \oplus A$ in ~ 12 seconds, while with the abstraction mechanism, it leads to a proof in ~ 260 seconds.



(a) Cactus plot of runtimes for varying number of abstractions and ways of providing them to Twee – as axioms or abstractions.

(b) Cactus plot of runtimes for varying weight factors (see Section 4).

Fig. 3: Cactus plots illustrating the effect of local abstractions. A given point in these plots corresponds to a particular problem P from Table 1. The y-value of a point corresponds to the time taken by Twee to find a proof of P when augmented with local abstractions, i.e. abstractions derived from an existing proof of P by default Twee. The x-value corresponds to the index of the problem when the problems are sorted by increasing runtime of Twee P augmented with the local abstractions. In other words, the points correspond to runtimes from line 7 of Algorithm 2.

Finally, we include Figure 3b which shows a cactus plot of abstraction-augmented runtimes with varying weight factors. A weight factor of about 0.5 (halving the cost of terms matching abbreviations) seems to work best. This aligns with the intuition that low weight factors are more aggressive and hence lead to more slowdowns (in particular, timeouts). However, some individual problems have the greatest speed-ups with lower weight factors.

6 Related Work

Our implementation of abstractions in Twee is an automatically controlled instance of the *weighting strategy* [14]. Weighting later influenced the work of Wos on the *resonance strategy* [23]. Resonators are clause patterns that bias clause selection toward shapes deemed promising by a human expert. This approach proved effective in difficult domains [3,8]. At around the same time Veroff introduced the *hint strategy* [19] as well as the *proof sketches* method [20], in which a prover first establishes an easier or related theorem and then reuses clauses from that proof as hints to guide the search for the harder goal. Our abstractions are in spirit the opposite of Veroff’s hints: hint clauses are key clauses which might be derived only once, whereas our abstractions are key term structures which appear over and over again.

Closely related to (and inspiration for) our use of STITCH is previous work on automated proof compression. Urban et al. [21] propose introducing new

definitions to shorten proofs found by saturation provers. Their goal is proof refactoring and compression, whereas we use compression primarily as a mechanism for discovering search-guiding patterns. Similarly, recent work by Wernhard and Zombori [22] explores grammar-based compression of Metamath proofs.

Related to the idea that internal proof structure can be mined to improve future search is work on learning-based guidance for saturation provers, such as ENIGMA [9,6]. However, these approaches rely on statistical models for clause selection, whereas our method extracts explicit syntactic abstractions that provide lightweight and interpretable guidance tailored to equational completion.

The automatic ranking of interesting derived results has been studied by Puzis et al. [16], who use automated reasoning to generate and filter potentially interesting theorems. Their work inspired our simple heuristic for ranking lemmas extracted from partial Twee proofs, and further exploration of their measures in our setting is an interesting direction for future work.

Other approaches to learning from related problems have been explored in automated reasoning. For example, the *Octopus* system [13] combines learning and parallel search, using information from easier problems to guide proofs of harder ones. This aligns with our domain abstraction approach, which is inspired by curriculum learning [4]: simpler problems in a domain provide reusable structural patterns for harder ones.

Automated theorem provers are increasingly used as genuine research tools in algebra to obtain mathematically interesting results. For instance, many results in quasigroup and loop theory have been obtained with the assistance of Prover9 and Otter[10,15]. In such settings, the usefulness of ATPs depends not only on finding proofs, but also on producing derivations that can be understood and reformulated in standard mathematical terms. Techniques such as proof sketches and proof simplification address this need. Our abstractions are related in spirit: the recurring term patterns identified by Twitch would ideally correspond to algebraically meaningful constructions, and guiding proof search toward these patterns may help produce proofs that are more structured and easier to analyse.

7 Limitations and Future Work

The system we have presented is in many ways quite simple-minded, and there are many ways in which it can be refined. Firstly, our domain construction is quite crude: we restrict to problems within the same TPTP theory sharing concrete symbols, and we do not attempt to automatically generate related or weakened conjectures in the style of proof sketches. Likewise, lemma scoring and term extraction from proofs are based on simple heuristics that could likely be refined as a path to more useful abstractions.

Further, abstraction selection is treated as a single-objective optimization problem focused primarily on runtime speedup. Alternative or multi-objective criteria — such as proof length reduction or robustness across strategies — may yield better guidance. It may also be fruitful to learn abstraction proposals directly from accumulated (problem, abstraction) training data, potentially

via language models. Despite these simplifications, we believe our results demonstrate that even a relatively simple abstraction pipeline can produce abstractions yielding substantial improvements and solve previously intractable problems, and that this suggests that automated, structure-based abstraction discovery is a promising direction for future research in equational theorem proving. We also note that for some problems, the best runtimes were obtained by providing both domain and partial proof abstractions. In particular, one more hard problem – LCL351-10 – was solved this way, where all other approaches failed, indicating potential of further benefits through such combination.

Also, the implementation of abstractions in Twee is currently quite brittle. The problem is that we only give bonuses to critical pairs that contain instances of the abstraction terms. However, Twee (in common with superposition provers) demodulates critical pairs, simplifying them using the generated rewrite rules. Thus it can easily happen that a term matching an abstraction gets rewritten to a term that does not match. For example, consider the abstraction $f(x, x)$. If f is also associative ($f(f(x, y), z) \rightarrow f(x, f(y, z))$) then the term $f(f(x, x), y)$ (which matches the abstraction) will be rewritten to $f(x, f(x, y))$ (which does not). We are experimenting with a critical pair-like mechanism for generating extra *derived abstractions* when an abstraction overlaps with a rewrite rule in such a way that rewriting destroys the abstraction. We are able to generate many extra abstractions this way, but the generated abstractions can very different from the user-provided abstractions and it is not clear if they respect the user’s intent.

Finally, we would like to explore generating not only abstractions but also hints. These are key steps that are needed in a proof, but may be difficult for the prover to find. An example of a good hint might be a rewrite rule which is large (hence is unlikely to be selected by Twee) but leads to a proof of a conjecture or important lemma.

8 Conclusion

We have shown that abstractions, automatically generated via compression of existing proofs, are a promising strategy for equational proving. Augmenting Twee with pre-learned domain abstractions allows us to prove several difficult conjectures and roughly halves the prover’s runtime for more typical conjectures.

Creating high-quality abstractions currently requires a supply of simpler problems within the same domain. In cases where these do not exist, we can also generate abstractions from a failed proof attempt, which are currently lower quality. Since typically only a few simple domain abstractions are needed, we hope that a more sophisticated extraction mechanism can solve this problem.

There is clearly much potential to refine the method, both how the abstractions are found and how they are exploited by the prover. Even so, we believe the results are promising and motivate further exploration of this idea.

Acknowledgments. This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice

Wallenberg Foundation, and by the Swedish Research Council (VR) grant 2025-06153, Semantically-guided theorem proving for mathematics.

References

1. Twitch experiment repository. https://github.com/WeAreDevo/ijcar26-twee_abstractions
2. Bachmair, L., Dershowitz, N., Plaisted, D.A.: Completion without failure. In: *Rewriting Techniques*, pp. 1–30. Academic Press (1989), <https://doi.org/10.1016/B978-0-12-046371-8.50007-9>
3. Beeson, M., Wos, L.: Finding proofs in tarskian geometry. *J. Autom. Reason.* **58**(1), 181–207 (Jan 2017). <https://doi.org/10.1007/s10817-016-9392-2>, <https://doi.org/10.1007/s10817-016-9392-2>
4. Bengio, Y., Louradour, J., Collobert, R., Weston, J.: Curriculum learning. In: *Proceedings of the 26th Annual International Conference on Machine Learning*. p. 41–48. ICML ’09, Association for Computing Machinery, New York, NY, USA (2009). <https://doi.org/10.1145/1553374.1553380>, <https://doi.org/10.1145/1553374.1553380>
5. Bowers, M., Olausson, T.X., Wong, L., Grand, G., Tenenbaum, J.B., Ellis, K., Solar-Lezama, A.: Top-down synthesis for library learning. *Proceedings of the ACM on Programming Languages* **7**(POPL), 1182–1213 (Jan 2023). <https://doi.org/10.1145/3571234>, <http://dx.doi.org/10.1145/3571234>
6. Chvalovský, K., Jakubův, J., Suda, M., Urban, J.: ENIGMA-NG: Efficient neural and gradient-boosted inference guidance for E. In: *Automated Deduction - CADE 27: 27th International Conference on Automated Deduction, Natal, Brazil, August 27–30, 2019, Proceedings*. pp. 197–215. Springer-Verlag, Berlin, Heidelberg (2019). https://doi.org/10.1007/978-3-030-29436-6_12, https://doi.org/10.1007/978-3-030-29436-6_12
7. Claessen, K., Smallbone, N.: Efficient encodings of first-order Horn formulas in equational logic. In: Galmiche, D., Schulz, S., Sebastiani, R. (eds.) *Automated Reasoning*. pp. 388–404. Springer International Publishing, Cham (2018)
8. Fitelson, B., Wos, L.: Missing proofs found. *Journal of Automated Reasoning* **27**(2), 201–225 (2001)
9. Jakubův, J., Urban, J.: Enigma: Efficient learning-based inference guiding machine. In: Geuvers, H., England, M., Hasan, O., Rabe, F., Teschke, O. (eds.) *Intelligent Computer Mathematics*. pp. 292–302. Springer International Publishing, Cham (2017)
10. Kinyon, M., Phillips, J.: Rectangular quasigroups and rectangular loops. *Computers & Mathematics with Applications* **49**(11), 1679–1685 (2005). <https://doi.org/https://doi.org/10.1016/j.camwa.2005.01.018>, <https://www.sciencedirect.com/science/article/pii/S0898122105001896>
11. Knuth, D.E., Bendix, P.B.: Simple Word Problems in Universal Algebras, pp. 342–376. Springer Berlin Heidelberg, Berlin, Heidelberg (1983). https://doi.org/10.1007/978-3-642-81955-1_23
12. McCune, W.: OTTER 3.3 reference manual. Tech. rep. (2003)
13. Newborn, M., Wang, Z.: Octopus: Combining learning and parallel search. *Journal of Automated Reasoning* **33**(2), 171–218 (2004)
14. Overbeek, R., McCharen, J., Wos, L.: Complexity and related enhancements for automated theorem-proving programs. *Computers & Mathematics with Applications* **2**(1), 1–16 (1976). [https://doi.org/https://doi.org/10.1016/0898-1221\(76\)90002-X](https://doi.org/https://doi.org/10.1016/0898-1221(76)90002-X), <https://www.sciencedirect.com/science/article/pii/089812217690002X>

15. Phillips, J.D., Stanovský, D.: Automated theorem proving in quasigroup and loop theory. *AI Commun.* **23**(2-3), 267–283 (Apr 2010)
16. Puzis, Y., Gao, Y., Sutcliffe, G.: Automated generation of interesting theorems. In: *FLAIRS*. pp. 49–54 (2006)
17. Smallbone, N.: Twee: An equational theorem prover. In: *CADE*. pp. 602–613 (2021)
18. Sutcliffe, G., Suttner, C.: The TPTP problem library. *Journal of Automated Reasoning* **21**(2), 177–203 (1998)
19. Veroff, R.: Using hints to increase the effectiveness of an automated reasoning program: Case studies. *Journal of Automated Reasoning* **16**(3), 223–239 (1996)
20. Veroff, R.: Solving open questions and other challenge problems using proof sketches. *Journal of Automated Reasoning* **27**(2), 157–174 (2001)
21. Vyskočil, J., Stanovský, D., Urban, J.: Automated proof compression by invention of new definitions. In: *International Conference on Logic for Programming Artificial Intelligence and Reasoning*. pp. 447–462. Springer (2010)
22. Wernhard, C., Zombori, Z.: Mathematical knowledge bases as grammar-compressed proof terms: Exploring metamath proof structures (2025), <https://arxiv.org/abs/2505.12305>
23. Wos, L.: The resonance strategy. *Computers & Mathematics with Applications* **29**(2), 133–178 (1995). [https://doi.org/https://doi.org/10.1016/0898-1221\(94\)00220-F](https://doi.org/https://doi.org/10.1016/0898-1221(94)00220-F), <https://www.sciencedirect.com/science/article/pii/089812219400220F>

A Experiment Parameters

There are various parameters that influenced the performance of Twitch and Twee when running the experiments. We list them here. General STITCH and Twee parameters:

- The number of abstractions STITCH is allowed to return.
- The maximum arity of the abstractions returned by STITCH.
- Whether to run Twee with (`flattening = True`) or without (`flattening = False`) goal flattening.
- The `weight-factor` parameter of Twee influencing the effect of abstractions.

Parameters specific to domain abstractions:

- The strategies to use during the local abstraction phase.
- The timelimit for Twee runs during the local abstraction phase.
- The threshold τ for determining good abstractions (see line 9 of Algorithm 1).
- Whether to run the final STITCH step in line 10 of Algorithm 1 to compress the good abstractions; or simply take all good abstractions as domain abstractions.

Parameters specific to partial proof abstractions:

- The amount of time t_{par} to run Twee for to generate the partial proof.
- The number k of top lemmas to take from the partial proof.

Throughout the experiments, the STITCH maximum arity parameter is fixed with the value 5. Similarly, the number of abstractions STITCH is allowed to return is fixed at 10.

To produce the domain abstractions used throughout the paper, the LOCALABSTRACTIONS phase consisted of running each domain problem (for 1000 seconds) for a fixed collection of strategies. If we identify a strategy with a pair (**flattening**, **weight-factor**), then the set of strategies we used is the product $\{True, False\} \times \{0, 0.2, 0.5, 0.7\}$.

In Figure 2a, we take the top 50 partial proof lemmas and have **weight-factor** = 0.2.

In Figure 2b, we have **weight-factor** = 0.2, $\tau = 0.2$, and the final STITCH step enabled.

To produce the results on the hard problems solved with partial proof abstractions, we ran Twee (with goal flattening set to **flattening**) for t_{par} seconds to generate the partial proof, and then took the top k lemmas from that proof to generate abstractions from. We then reran Twee (with goal flattening set to **flattening**, and **weight-factor** = 0.2) on the hard problem augmented with the partial proof abstractions for the remaining $(1000 - t_{\text{par}})$ seconds. We did this for all $(\text{flattening}, t_{\text{par}}, k) \in \{True, False\} \times \{50, 150, 500\} \times \{50, 100\}$.

To produce the results on the hard problems solved with domain abstractions, we iterated over a spread of 8 combinations of the relevant parameter values, which each have varying success on different problems. Specifically, we tested the following parameter combinations for each problem:

- **flattening** = *True*; **abstraction-weight-factor** = 0.2; $\tau = 0.7$; final STITCH step enabled.
- **flattening** = *False*; **abstraction-weight-factor** = 0; $\tau = 0.2$; final STITCH step disabled.
- **flattening** = *False*; **abstraction-weight-factor** = 0; $\tau = 0.7$; final STITCH step disabled.
- **flattening** = *True*; **abstraction-weight-factor** = 0; $\tau = 0.5$; final STITCH step disabled.
- **flattening** = *False*; **abstraction-weight-factor** = 0.5; $\tau = 0.5$; final STITCH step enabled.
- **flattening** = *False*; **abstraction-weight-factor** = 0.2; $\tau = 0.2$; final STITCH step enabled.
- **flattening** = *False*; **abstraction-weight-factor** = 0.5; $\tau = 0.5$; final STITCH step enabled.
- **flattening** = *True*; **abstraction-weight-factor** = 0.2; $\tau = 0.2$; final STITCH step disabled.

Here, `flattening` refers to the goal flattening parameter of Tweep when run on the hard problem augmented with the domain abstractions.

In Figures 3a and 3b, all runs were done with `flattening = False`. In Figure 3a, the `abstraction-weight-factor` is set to 0.5 (this has no effect on the definitional axiom runs).

B Hardware

All experiments were run on an AMD Zen 4 EPYC 9354 core.

C Hard Problems Solved

The following hard problems were solved with the help of abstractions derived by Twitch. Recall that the ratings are those as of TPTP v9.2.1. For the full list of successful runs and their associated hints and experiment parameter values, see the `json` files in the `data/experiments/hard_successes` directory of the code repository: https://github.com/WeAreDevo/ijcar26-tweep_abstractions.

C.1 Partial Proof Abstractions

Using partial proof abstractions:

- GRP723-1.p (Rating: 1.00) – Best time: 418.47s
- LAT075-1.p (Rating: 1.00) – Best time: 141.11s
- LCL375-10.p (Rating: 1.00) – Best time: 399.95s
- LCL395-10.p (Rating: 1.00) – Best time: 376.37s
- LCL225-10.p (Rating: 0.96) – Best time: 11.55s
- LCL245-10.p (Rating: 0.96) – Best time: 114.44s
- LCL289-10.p (Rating: 0.96) – Best time: 130.55s
- LCL348-10.p (Rating: 0.96) – Best time: 11.56s
- ROB031-2.p (Rating: 0.96) – Best time: 12.78s
- ROB035-1.p (Rating: 0.96) – Best time: 12.30s
- LAT400-1.p (Rating: 0.91) – Best time: 698.88s

C.2 Domain Abstractions

Using domain abstractions:

- GRP672-11.p (Rating: 1.00) – Best time: 510.31s
- GRP723-1.p (Rating: 1.00) – Best time: 708.57s
- GRP732-1.p (Rating: 1.00) – Best time: 549.19s
- LAT075-1.p (Rating: 1.00) – Best time: 31.94s
- LCL304-10.p (Rating: 1.00) – Best time: 119.69s
- LCL331-10.p (Rating: 1.00) – Best time: 40.88s

- LCL375-10.p (Rating: 1.00) – Best time: 119.14s
- LCL395-10.p (Rating: 1.00) – Best time: 522.33s
- ROB032-2.p (Rating: 1.00) – Best time: 41.19s
- ROB034-1.p (Rating: 1.00) – Best time: 42.35s
- LCL225-10.p (Rating: 0.96) – Best time: 295.57s
- LCL245-10.p (Rating: 0.96) – Best time: 125.09s
- LCL289-10.p (Rating: 0.96) – Best time: 105.07s
- LCL339-10.p (Rating: 0.96) – Best time: 247.18s
- LCL348-10.p (Rating: 0.96) – Best time: 31.57s
- ROB031-2.p (Rating: 0.96) – Best time: 33.79s
- ROB035-1.p (Rating: 0.96) – Best time: 32.73s
- LAT400-1.p (Rating: 0.91) – Best time: 160.89s

Using a hand-picked set of domain problems:

- GRP677-1.p (Rating: 1.00) – Best time: 180s

Using domain abstractions together with partial proofs:

- LCL351-10.p (Rating: 1.00) – Best time: 200s